challenge_filters_pagination

Helix OTA — `challenge_filters_pagination.sh` companion guide

Revision: 1 **Last modified:** 2026-06-10T10:10:00Z

External user guide (Constitution §11.4.18) for `tests/e2e/challenge_filters_pagination.sh`.

Overview

`challenge_filters_pagination.sh` is an autonomous, anti-bluff end-to-end challenge (Constitution §11.4 / §11.4.27 / §11.4.69 / §11.4.98) that covers the two just-shipped query features of the Helix OTA control plane:

1. **Per-device telemetry filters** — `GET /api/v1/devices/{id}/telemetry?event=&since=&until=&limit=&cursor=`
2. **Group + group-members cursor pagination** — `GET /api/v1/groups?limit=&cursor=` and `GET /api/v1/groups/{id}/members?limit=&cursor=`, both returning `next_cursor`.

Unlike `challenge_operational.sh` (which black-box-drives an *already-running* server via `--base-url`), this script is **fully self-hosting**, mirroring the `pipeline_signed.sh` harness: it **builds** `ota-server` from source, **boots** it with ephemeral in-memory configuration, **seeds** real state through the real REST API, **drives** the two features over real HTTP (`curl` + `jq`), and **tears the server down** on every exit path. There are **no mocks** of the system under test and **no manual steps**.

Prerequisites

- `bash`, `go`, `curl`, `jq` on `PATH`.
- No database (the server uses its in-memory repository).
- No admin password to supply — the script mints an ephemeral admin password and token secret for its own run; nothing is committed (§11.4.10).

If any of `go`/`curl`/`jq` is missing, or `go build` fails, the script exits `3` (SKIP-with-reason) rather than fabricating a PASS (§11.4.3 / §11.4.6).

Usage examples

Run it directly (builds + boots its own server on port 8096):

```
bash tests/e2e/challenge_filters_pagination.sh
```

Pick a different port (e.g. to avoid a clash):

```
HELIX_PORT=8097 bash tests/e2e/challenge_filters_pagination.sh
# or
bash tests/e2e/challenge_filters_pagination.sh --port 8097
```

Reuse a pre-built server binary (skips the `go build`):

```
( cd server && go build -o /tmp/ota-server ./cmd/ota-server )
bash tests/e2e/challenge_filters_pagination.sh --server-bin /tmp/
        ota-server
```

Inspect the captured run (tee'd live to this file):

```
cat tests/e2e/FILTERS_PAGINATION_EVIDENCE.txt
```

Inputs

Input	Default	Meaning
`--port` / `$HELIX_PORT`	`8096`	TCP port for the self-hosted server
`--server-bin`	*(build into a temp dir)*	pre-built `ota-server` binary
`$HELIX_FILTERS_EVIDENCE`	`tests/e2e/FILTERS_PAGINATION_EVIDENCE.txt`	evidence file path

Outputs

- Human-readable `[PASS]`/`[FAIL]`/`[SKIP]` lines on stdout **and** in the evidence file, ending with `== summary: N passed, M failed, K skipped ==` and a `RESULT: PASS|FAIL` line.
- Exit code: `0` only if every hard assertion passed; non-zero on any mismatch; `3` on a missing prerequisite / unbuildable server (SKIP, never a false PASS).

Side-effects

Builds + starts + stops exactly one `ota-server` (in-memory repo — no database, no host state touched); frees the port on every exit path via `trap cleanup EXIT INT TERM`; writes the evidence file under `tests/e2e/`. Ephemeral build output and the server log live in an `mktemp` dir that is `rm -rf`'d on exit.

Internal behaviour

1. **Build + boot** — `go build ./cmd/ota-server` into a temp dir, start it with `HELIX_PORT` / ephemeral `HELIX_ADMIN_*` / `HELIX_TOKEN_SECRET`, poll `/healthz` until ready (~10 s budget).
2. **Login** — `POST /api/v1/auth/login` → bearer token; an unauthenticated `GET /telemetry/overview` must return **401** (auth-enforcement anti-bluff).
3. **Seed telemetry** — register a device, then ingest **six** events via `POST /api/v1/client/telemetry` at distinct types (`download_started`, `installing`×2, `verifying`, `success`, `failure`) with ascending RFC3339 timestamps. The ingest acknowledgement `.accepted` must equal `6` — so a later green filter result over *un-ingested* data is impossible.
4. **Feature 1 assertions** — `?event=` returns only the matching type; a bogus `?event=` → 400; `?since/?until` is an **inclusive** window (both boundary timestamps present, none outside); combined event+window narrows correctly; an inverted window → empty + null `next_cursor`; `?limit=2` cursor walk visits all six events with no overlap/gap.
5. **Feature 2 assertions** — create 3 groups; `?limit=2` page1 = 2 items + non-null `next_cursor`, page2 = remaining 1 + null `next_cursor`, union = all 3 distinct groups (no overlap/gap). Register 3 devices, batch-add them to a group, and paginate members the same way (each item carries a non-empty `added_at`). Validate `limit`/`cursor` bounds (`limit` out of `[1,200]` and a malformed/negative `cursor` → 400).

Anti-bluff guarantees (each HARD-FAILs the challenge)

- a bogus `?event=` that returned 200 (silent-empty) instead of 400;
- an `?event=` filter result that included an event of a different type;
- a `since/until` window that included an out-of-window event, or excluded a boundary event (inclusive bounds);
- page1+page2 that overlap, drop, or duplicate any id;
- a `next_cursor` that is non-null on the genuinely-last page;
- a 401 on a protected route while seeded with a valid token.

Captured evidence (real run)

```
== summary: 50 passed, 0 failed, 0 skipped ==
RESULT: PASS
```

Re-run end-to-end any number of times — verified twice consecutively with an identical `50 passed, 0 failed, 0 skipped` (§11.4.50 deterministic consistency, §11.4.98 full-automation). Full transcript: `tests/e2e/FILTERS_PAGINATION_EVIDENCE.txt`.

Related scripts

- `tests/e2e/pipeline_signed.sh` — the self-hosting boot harness this script mirrors (signed artifact pipeline).
- `tests/e2e/challenge_operational.sh` — black-box operational + rollout challenge against an already-running server.

- `tools/helixqa/banks/helix_ota.yaml` — the HelixQA bank; challenge `HOTA-FILTERS-PAGINATION` dispatches this script.

Source references

- `server/internal/api/handlers_telemetry.go` — `handleDeviceTelemetry` (`event` / `since` / `until` / `limit` / `cursor`).
- `server/internal/api/handlers_group.go` — `parsePage`, `handleListGroups`, `handleListGroupMembers`.
- `server/internal/api/handlers_client.go` — `handleClientTelemetry` (ingest).

Last verified: 2026-06-10